# Project Report

GTSRB Traffic-Sign Recognition

Akshit Mehta


## ABSTRACT

This report presents a reproducible TensorFlow/Keras pipeline for classifying the 43 categories in the German Traffic Sign Recognition Benchmark. The implementation downloads and verifies the full archive, analyzes its statistical structure, prepares grouped training and validation data, trains two neural architectures, evaluates the official test split, and exposes single-image prediction. A static dashboard and Colab notebook make the workflow easy to inspect and rerun.


## 1. INTRODUCTION

Traffic-sign recognition is a useful computer-vision benchmark because the classes are semantically meaningful yet visually similar. Signs can differ by shape, text, color, border, and symbol, while the captured images vary in illumination, viewpoint, scale, and background. A robust project therefore needs more than a model definition: it needs a trustworthy dataset boundary, leakage-aware validation, diagnostics, and clear reporting.

## 2. REQUIREMENTS AND SCOPE

The project targets classification of an already-cropped sign image. It does not locate signs in a full scene, issue driving instructions, or make safety-critical decisions. The implementation exposes model choice, image size, seed, epochs, batch size, learning rate, sample limits, and output directory through command-line options.

## 3. DATASET

The complete GTSRB classification archive is downloaded from the Zenodo record 13741936. The published MD5 checksum is verified before extraction. The extracted package contains the 39,209 labeled training images in class directories, the 12,630 official test images, and the test labels. All expected class IDs 0 through 42 are checked.

| Dataset property | Value |
|---|---|
| Training split | 39,209 images |
| Official test split | 12,630 images |
| Label space | 43 classes, IDs 0–42 |
| Training class minimum | 210 images |
| Training class maximum | 2,250 images |
| Archive integrity | MD5 verified |

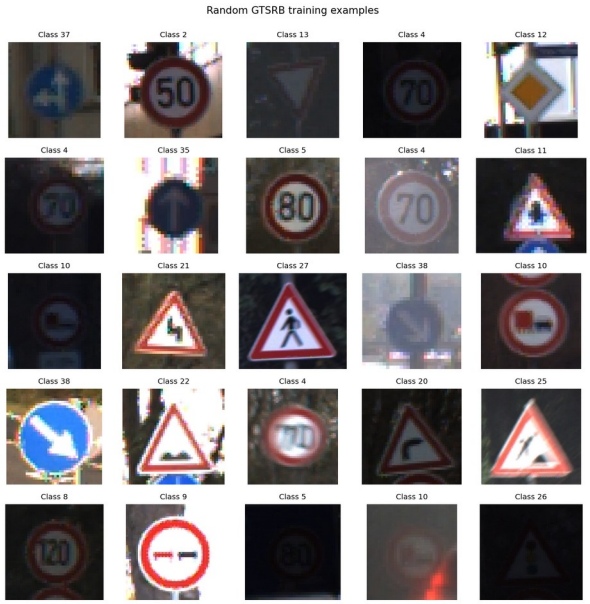



*Figure 1. Representative examples from the training set.*

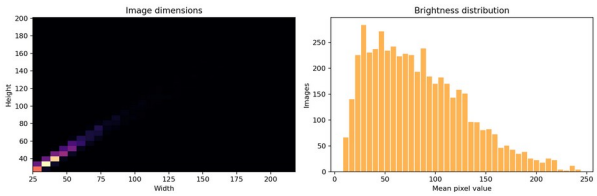



*Figure 2. Sampled image dimensions and brightness distributions.*

## 4. EXPLORATORY DATA ANALYSIS

The class distribution is uneven, so raw accuracy alone is insufficient. Macro F1 is reported alongside accuracy to give every class equal weight. Sampled image dimensions show that the source images are not uniform, which motivates consistent resizing. Brightness and RGB statistics confirm meaningful illumination and color variation.

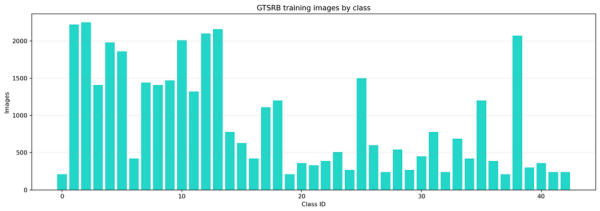



*Figure 3. The 43-class training distribution.*

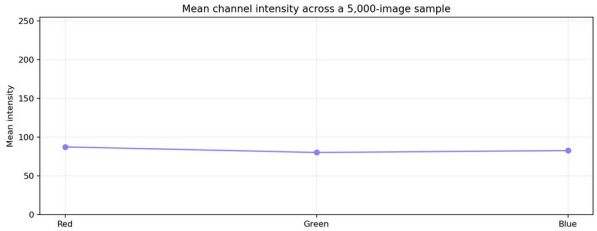



*Figure 4. Mean RGB channel intensity over a 5,000-image sample.*

## 5. DATA PREPARATION AND LEAKAGE CONTROL

The loader supports both the Zenodo package layout and the official GTSRB directory layout. Training labels are inferred from class directories; test labels are read from Test.csv. Related images are assigned sequence-like group identifiers derived from their source paths and grouped before the validation split. This reduces the chance that near-duplicate frames from the same sequence appear on both sides of validation.

Images are resized and normalized to [0, 1]. The augmentation pipeline applies moderate rotation, translation, zoom, brightness, and contrast changes only during training. The official test set remains untouched until final evaluation.

## 6. MODELS

### 6.1 Custom CNN baseline

The baseline uses stacked convolutional blocks with batch normalization, max pooling, dropout, and a dense classification head. It is intentionally compact so its behavior is easy to explain and its CPU smoke test is fast.

### 6.2 MobileNetV2 comparison

MobileNetV2 provides a lightweight transfer-learning architecture. The project supports pretrained ImageNet weights when available and a no-pretrained option for offline or smoke-test execution. Its depthwise-separable convolutions provide a useful accuracy/compute comparison with the custom baseline.

## 7. TRAINING PROTOCOL

| Setting | Custom CNN | MobileNetV2 |
|---|---|---|
| Input size | 64 × 64 | 128 × 128 |
| Seed | 42 | 42 |
| Epochs in recorded run | 30 | 25 |
| Batch size | 128 | 128 |
| Validation | Grouped split | Grouped split |
| Hardware | Colab GPU | Colab GPU |
| Callbacks | Early stopping, LR reduction, checkpoint | Early stopping, LR reduction, checkpoint |

Each run saves the best Keras model, a JSON history, training curves, a classification report, and a confusion matrix under artifacts/models. Repeated execution is deterministic at the seed and data-split level, subject to TensorFlow and hardware determinism.

## 8. LITERATURE SURVEY AND DESIGN RATIONALE

The project design is grounded in verified primary research and official documentation. Stallkamp et al. established GTSRB as a realistic multi-class traffic-sign benchmark with variation in illumination, weather, distance, occlusion, and rotation; this motivates the 43-class scope, EDA, augmentation, and official-test evaluation. Sandler et al. introduced MobileNetV2's efficient inverted-residual architecture; this motivates its use as a lightweight transfer-learning comparison. Kingma and Ba introduced Adam, used here as a practical baseline optimizer. Selvaraju et al. introduced Grad-CAM, used here for qualitative inspection of whether predictions focus on plausible sign regions. Official scikit-learn documentation supports GroupShuffleSplit for keeping related groups together during validation, while TensorFlow and Keras documentation supports streaming and prefetched tf.data pipelines, training-time augmentation, early stopping, learning-rate reduction, and checkpoint saving.

| Reference | Verified contribution | Implementation role |
|---|---|---|
| Stallkamp et al., IJCNN 2011 | GTSRB benchmark definition and real-world visual variation | Dataset boundary and evaluation |
| Sandler et al., CVPR 2018 | Efficient MobileNetV2 architecture | Transfer-learning comparison |
| Kingma and Ba, ICLR 2015 | Adaptive Adam optimizer | Training optimizer |
| Selvaraju et al., ICCV 2017 | Gradient-based visual explanations | Grad-CAM diagnostic |
| scikit-learn / TensorFlow / Keras documentation | Grouped splitting, streaming input, augmentation, callbacks | Validation and training pipeline |

The detailed source-to-code mapping and verified links are provided in docs/literature_survey.md. These sources justify design choices; they do not imply that the present configuration is optimal. The optional YOLO code remains a detector scaffold because the GTSRB classification archive does not provide full-scene bounding-box labels.

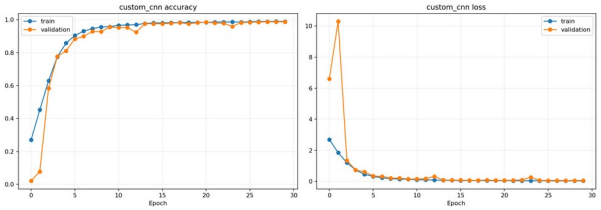



*Figure 5. Custom CNN training and validation curves.*

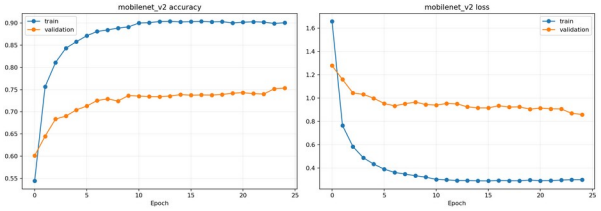



*Figure 6. MobileNetV2 training and validation curves.*

## 9. RESULTS

| Model | Test accuracy | Macro F1 |
|---|---|---|

| | | |
|---|---|---|
| Custom Cnn | 97.12% | 95.14% |
| Mobilenet V2 | 76.18% | 68.23% |

In these full-data Colab runs, the custom CNN is ahead on both overall accuracy and macro F1. This result is substantially stronger than the short local baseline, while still remaining a reproducible project benchmark rather than a maximum-performance claim.

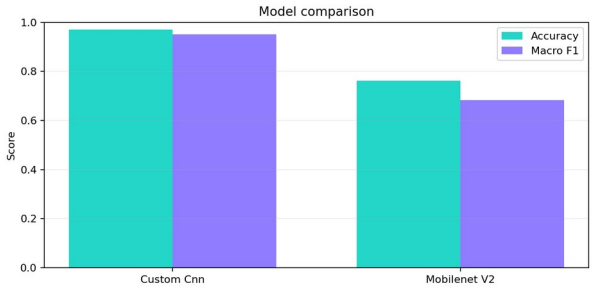



*Figure 7. Test accuracy and macro F1 comparison.*

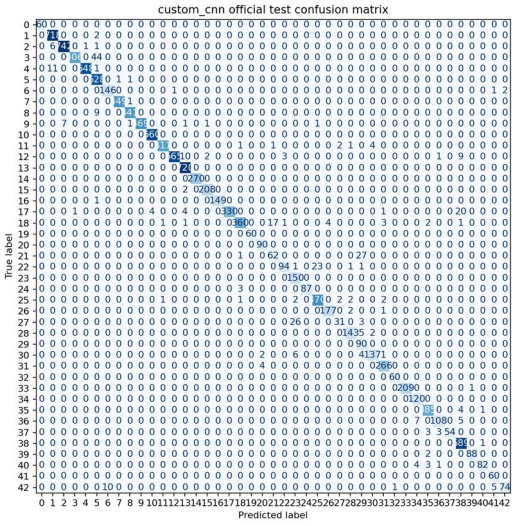



*Figure 8. Custom CNN confusion matrix.*

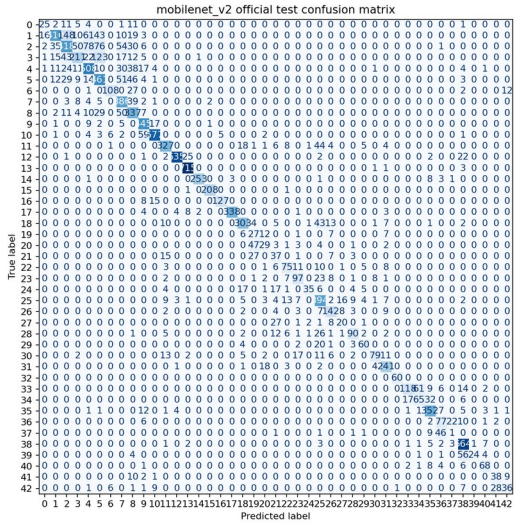



*Figure 9. MobileNetV2 confusion matrix.*

## 10. ERROR ANALYSIS AND LIMITATIONS

The confusion matrices and per-class reports expose which signs remain difficult to separate. Likely sources include visually similar speed-limit classes, class imbalance, small or blurred signs, and the short demonstration training budgets. The pipeline does not yet include class-balanced loss, systematic hyperparameter search, calibration, test-time augmentation, or human review of mislabeled examples.

The classifier assumes a cropped sign image. It should not be used as a standalone driving or safety system. A deployment-oriented extension would require object detection, temporal smoothing, uncertainty thresholds, out-of-distribution checks, and extensive testing under real-world conditions.

## 11. REPRODUCIBILITY AND USAGE

- python scripts/download_dataset.py
- python scripts/run_eda.py
- python -m traffic_sign_recognition.train --model custom_cnn
- python -m traffic_sign_recognition.train --model mobilenet_v2
- python -m traffic_sign_recognition.evaluate --model-path artifacts/models/mobilenet_v2/model.keras
- python -m traffic_sign_recognition.predict --model-path artifacts/models/mobilenet_v2/model.keras --image path/to/sign.png

## 12. CONCLUSION AND FUTURE WORK

The project demonstrates a complete and auditable traffic-sign recognition workflow: the source archive is verified, data assumptions are visualized, two models are trained, results are measured, and inference is packaged for reuse. Future work should use matched multi-epoch GPU training, class-aware sampling or loss functions, richer error analysis, and a detector-plus-classifier system for full road scenes.

## REFERENCES

Stallkamp, J., Schlipsing, M., Salmen, J., and Igel, C. (2011). The German Traffic Sign Recognition Benchmark: A multi-class classification competition. Proceedings of IJCNN.

GTSRB public dataset distribution and Zenodo mirror: https://zenodo.org/records/13741936

TensorFlow/Keras documentation: https://www.tensorflow.org/